

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 3 — Esercitazione: una dashboard Artifact da dati sintetici anonimizzati

Prima di cominciare

Prima di cominciare

Il laboratorio del Modulo 3, in tre esercitazioni; e quattro controlli prima di M3-E01

- Il **laboratorio**: una settimana di ticket dell'assistenza clienti di **CiaspoLink**, l'operatore inventato del corso, dal file con i **dati personali** a una **dashboard**, in tre esercitazioni
- **M3-E01**, in **Claude Code**: anonimizzare la settimana; Claude scrive lo script senza aprire il file `m3_e2_anonimizzare/M3-E01_anonimizzare_ticket/`
- **M3-E02**, in **claude.ai**: la dashboard dal file anonimo, prima i numeri e poi l'artifact `m3_e3_dashboard/M3-E02_dashboard_settimana/`
- **M3-E03**, in **Claude Code**: la stessa dashboard, come pagina HTML locale `m3_e4_dashboard_locale/M3-E03_dashboard_locale/`
- Le cartelle stanno in `m3_funzionalita_claude/s2_funzionalita_claude/` del repository delle esercitazioni; chi non finisce M3-E01, o non ne passa la verifica, trova nelle cartelle di M3-E02 e M3-E03 una copia di riferimento del file anonimo
- Il ciclo di ogni esercitazione: **prepara** (`tools/prepara_esercizio.sh` e l'identificativo), **lavora** con i passi del README, **verifica** (`tools/verifica.sh`: 0 fatto, 1 non ancora), se serve **ricomincia** (`tools/reset_esercizio.sh`)

Prima di cominciare

Il laboratorio del Modulo 3, in tre esercitazioni; e quattro controlli prima di M3-E01

```
# nel terminale Linux o WSL 2, nella cartella esercitazioni
git pull                        # gli elementi del Modulo 3
claude --version                # 2.1.284 o successiva
tools/verifica_ambiente.sh      # deve finire con: Tutto pronto
claude doctor                   # se qualcosa non torna
```

- `claude --version`: **2.1.284 o successiva** — le sessioni del laboratorio in Claude Code girano su **Sonnet**, e con le versioni precedenti l'alias `sonnet` punta a un modello più vecchio
- In Claude Code, `/status`: l'accesso attivo dev'essere il vostro **abbonamento Pro**
- **Nessuna** `ANTHROPIC_API_KEY`: se c'è, Claude Code la usa al posto dell'abbonamento (con `claude -p`, sempre), e l'API non è nel piano Pro
- `tools/verifica_ambiente.sh` **tutto verde**: controlla anche la chiave, senza stamparla, e Python; `claude doctor` è una diagnostica in sola lettura

I dati della settimana

Sintetici, ma trattati come personali

- `dati/ticket-settimana.csv`: l'esportazione del sistema di ticketing dell'assistenza clienti di CiaspoLink per una settimana — **343 segnalazioni**, una per riga, **21 colonne**
- Il **dizionario delle colonne** è `dati/README.md`: che cosa contiene ogni colonna, e con quali valori. In tre gruppi:

il cliente	`id_cliente`, `tipo_cliente`, `nome_cliente`, `telefono`, `email`, `comune`, `id_linea`
la segnalazione	`id_ticket`, `canale`, `tecnologia`, `categoria`, `priorita`, `oggetto`, `descrizione`
come è andata	`aperto_il`, `chiuso_il`, `stato`, `team`, `minuti_prima_risposta`, `minuti_risoluzione`, `soddisfazione`

- Dati **personali inventati**, messi apposta: nomi, telefoni, e-mail, indirizzi e, nelle descrizioni, familiari, recapiti, codici, un'informazione sulla salute — da **trattare come veri**, e da anonimizzare prima di analizzarli con strumenti esterni
- E le imperfezioni di un sistema vero: **campi vuoti**, lo stesso valore **scritto in modi diversi**, **segnalazioni doppie** — il profilo della settimana le conta, il file anonimo non le corregge

Perché Claude non apre il file grezzo

Ciò che Claude Code legge entra nella conversazione: il file lo legge lo script

- Claude Code gira sul vostro computer, ma per usare il modello **manda in rete** prompt e risposte; ciò che passa da uno strumento, contenuti dei file compresi, resta su disco in una **trascrizione in chiaro**, 30 giorni per impostazione predefinita. E le condizioni per i consumatori valgono anche in Claude Code, con il Pro
- Così, in M3-E01, Claude **scrive lo script** dal dizionario delle colonne, e il file grezzo lo legge **lo script**; il controllo dice quanti dati personali restano e dove, **mai** i valori. Ciò che Claude legge del file anonimo, però, entra nella conversazione, con ciò che vi è rimasto
- La regola `deny` di `Read` sul file, nelle impostazioni dello spazio di lavoro (le regole si gestiscono con `/permissions`), ferma gli strumenti di Claude per i file, in ogni modalità; non un comando che lo legge senza nominarlo, né un programma che lo apre da sé: è una **guardia**, non un muro
- Togliere o anonimizzare i dati personali **prima di caricare**: è la risposta giusta dell'esempio 3 della guida all'esame Claude Certified Associate

Un ambiente verificato e un dato anonimo prima di ogni strumento.

Un limite in comune

Che cosa pesa sul limite del piano, e quanto pesa il laboratorio

- Un solo **limite d'uso**: claude.ai, Claude Code e l'app desktop attingono allo stesso; in Claude Code, `/usage` ne mostra le barre
- Incidono sul limite gli **artifact**, il **codice** eseguito e i **file** creati, strumenti come Research e la web search, il **modello** e l'**effort**; un compito agentico lungo, di solito, pesa più di una domanda veloce
- Conta la **lunghezza**, anche dei file allegati; il contenuto di un progetto, in cache, pesa meno quando lo riusate
- L'impronta del laboratorio: **due sessioni** di Claude Code su Sonnet, effort medium, senza subagent (M3-E01, M3-E03), e **una chat** in claude.ai con il file anonimo, il codice e un artifact (M3-E02) — proprio ciò che incide sul limite. Controlli e verifiche sono script, senza chiamate al modello